

RISC - V Verification Project

Index / Table of Contents :

- 1** Project Overview
- 2** RISC-V ISA & CPU Concept
- 3** RISC-V Interface Signals
- 4** Features Verified
- 5** UVM Verification Architecture
- 6** Verification Flow
- 7** Testbench Components
- 8** Final Verdict
- 9** Conclusion

Project Overview

This project verifies a RISC-V CPU core using SystemVerilog and UVM methodology. The verification environment validates instruction execution, register behavior, memory access, and control flow of the processor.

The following aspects are verified:

- Instruction fetch and decode
- Program Counter (PC) update logic
- Register file read/write correctness
- ALU operations (ADD, ADDI)
- Load/Store memory operations
- Branch and Jump instructions
- Architectural state consistency
- Reset behavior
- Directed + stress test scenarios
- Functional coverage for instruction types

This verification follows industry-standard CPU verification methodology, similar to real SoC processor validation environments.

2 RISC-V ISA & CPU Concept

RISC-V is an **open-standard Reduced Instruction Set Computer (RISC)** architecture.

i. CPU Execution Model

Each instruction goes through:

1. **Fetch** – instruction read from instruction memory
2. **Decode** – opcode, registers, immediate extraction
3. **Execute** – ALU / branch decision
4. **Memory** – load/store access
5. **Writeback** – register update

ii. Instruction Types Verified

Instruction Type	Example	Function
R-type	ADD	Register-to-register ALU
I-type	ADDI	Immediate ALU
Load	LW	Load from memory
Store	SW	Store to memory
Branch	BEQ	Conditional branch
Jump	JAL	Unconditional jump

iii. Program Counter (PC) Behavior

- Normally increments by +4
- Modified by branch/jump instructions
- Verified cycle-by-cycle in scoreboard

3 RISC-V Interface Signals

Signal	Direction	Description
clk	Input	System clock
reset	Input	Synchronous reset
pc_out	Output	Current Program Counter
imem_rdata	Input	Instruction fetched
dmem_addr	Output	Data memory address
dmem_wdata	Output	Store data
dmem_rdata	Input	Load data
dmem_we	Output	Write enable
dmem_wstrb	Output	Byte enable
regfile	Internal	Register file state

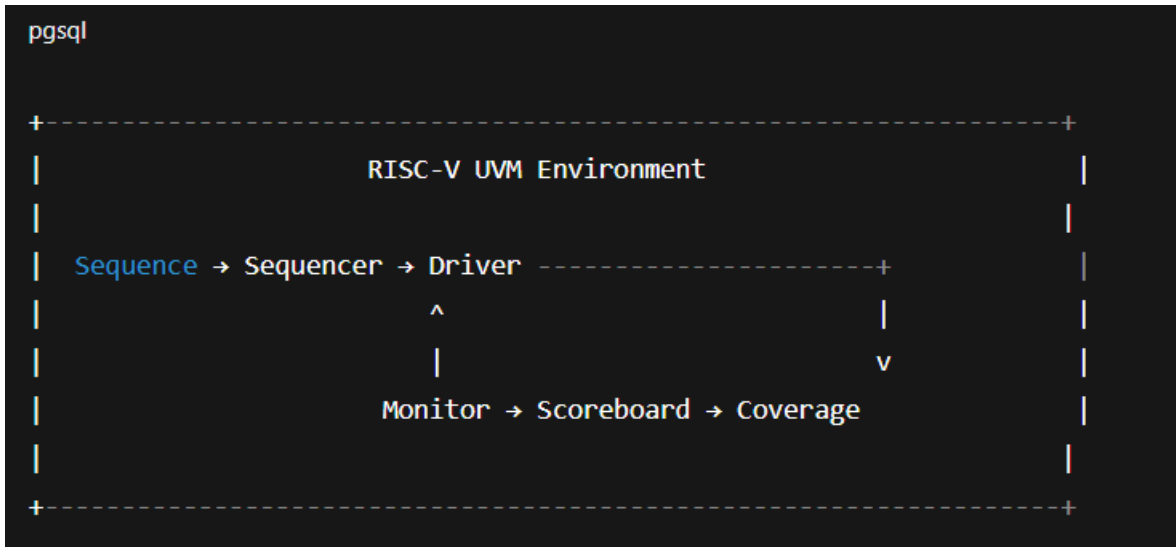


Features Verified

Instruction fetch correctness :

- Opcode decoding
- Register read/write correctness
- x0 register remains hard-wired to zero
- ALU result accuracy
- Immediate sign-extension logic
- Load/Store address calculation
- Memory data correctness
- Branch decision logic
- Jump target calculation
- PC update sequencing
- Reset behavior
- Back-to-back instruction execution
- RAW dependency scenarios
- No X-propagation in architectural state

5 UVM Verification Architecture



Components:

- Sequence Item – instruction, PC
- Sequences – reset, rtype, itype, load, store, branch, jump
- Driver – drives instructions into DUT
- Monitor – observes instruction & memory activity
- Scoreboard – reference CPU model
- Coverage – instruction & execution coverage
- Agent – driver + monitor
- Environment – agent + scoreboard + coverage
- Tests – directed + stress tests

6 Verification Flow

Phase	Test Type	Purpose
Smoke	Reset + NOP	Connectivity & sanity
Directed	Instruction tests	Functional correctness
Load/Store	Memory tests	Address & data integrity
Control Flow	Branch/Jump	PC update correctness
Stress	Back-to-back ops	Pipeline & dependency
Coverage	Functional	Scenario completeness
Regression	Multi-seed	Stability & sign-off

7 Testbench Components

7.1 Sequence Item (riscv_txn)

Contains:

Instruction word

Program Counter

Decoded fields (rs1, rs2, rd – optional)

7.2 Driver

Drives instructions to DUT

Synchronizes with clock

Handles reset sequencing

Ensures correct instruction timing

7.3 Monitor

Samples:

PC value

Instruction word

Memory read/write signals

Publishes:

Instruction transactions

Memory transactions

7.4 Scoreboard

Implements reference RISC-V architectural model:

```
exp_regs[rd] = exp_regs[rs1] + exp_regs[rs2];
```

```
exp_pc += 4;
```

Checks:

- PC correctness
- Register write correctness
- Memory load/store correctness
- Architectural state consistency

7.5 Coverage

Coverpoints:

- Instruction opcode
- Instruction type
- Register usage
- Load vs Store
- Branch taken vs not taken

Cross Coverage:

- opcode × instruction type
- register × operation

7.6 Assertions

Assertions ensure:

PC alignment

No invalid memory access

Correct reset behavior

8. Final Verdict

Directed Tests + Stress Tests + Scoreboard + Coverage = Complete RISC-V CPU Verification Sign-off

Scoreboard ensures functional correctness.

Coverage ensures scenario completeness.

Tests ensure real-world execution confidence. This is industry-standard CPU verification flow.

9. Conclusion

This RISC-V verification project validates complete CPU functionality using:

- UVM-based stimulus
- Reference-model scoreboard
- Functional coverage
- Directed + stress tests

It demonstrates strong understanding of CPU architecture, UVM methodology, and industry-level verification practices.

RISC – V Concepts

1 CPU Memory & PC Mechanism

A CPU executes a program by repeatedly:

1. Fetching an instruction
2. Executing it
3. Updating architectural state

Three core concepts enable this:

Program Counter (PC)

Instruction Memory (I-Mem)

Data Memory (D-Mem)

👉 Verification checks **what the CPU exposes architecturally**, not how it is implemented internally.

2 Program Counter (PC)

The Program Counter is a CPU register that holds the address of the next instruction to execute.

Functions of PC

1. Points to the current instruction address
2. Sends address to Instruction Memory
3. Updates every instruction

PC Update Rules (RV32I)

- Normal instruction $\rightarrow PC = PC + 4$
- Branch taken $\rightarrow PC = \text{branch_target}$
- Jump $\rightarrow PC = \text{jump_target}$

Location

- Inside CPU core (register)
- Not visible externally in real silicon

DV View of PC :

Why PC Matters in Verification

PC controls:

Instruction order

Control flow correctness

Branch & jump behavior

DV Checks :

PC increments by 4

Correct branch target

Correct jump target

No off-by-4 errors

Analogy :

PC = finger pointing to the next recipe card in a cookbook

3 Instruction Memory (I-Mem)

What is Instruction Memory? Memory that stores machine instructions.

Function

1. Receives address from PC
2. Returns instruction to CPU

Real CPU Location

L1 Instruction Cache (fast)

DRAM (slow)

DV Perspective of I-Mem : There is **NO real instruction memory** in the testbench.

How DV Models I-Mem

CPU drives instruction address

Testbench responds with instruction data

DV Check : ✓ Instruction returned matches expected instruction for that PC

Analogy : I-Mem = library of recipe cards

Data Memory (D-Mem)

What is Data Memory? Memory that stores **program data** (variables, arrays, results).

Operations

- Load → read data
- Store → write data

Control Signals

- dmem_addr → address
- dmem_we → write enable
- dmem_wdata → write data
- dmem_rdata → read data

Real CPU Location

- L1 Data Cache
- DRAM

DV Perspective of D-Mem

Key Rule: DV does NOT peek inside memory arrays.

DV only observes the memory interface: CPU → dmem_* → Memory

DV Checks

Correct address accessed

Correct data written

Correct data loaded

Analogy : D-Mem = pantry storing ingredients used in recipes

Takeaway (PC + I-Mem + D-Mem)

- PC drives instruction fetch
- I-Mem provides instructions
- D-Mem stores program data
- DV checks architectural behavior, not internal RAMs

5 Program Counter Output (pc_out)

What is pc_out? logic [31:0] pc_out;

A verification-visible copy of the internal PC

Exported only for DV

Why Expose pc_out?

PC is invisible in real silicon, but DV needs it to:

- Track instruction sequence
- Validate control flow
- Debug branch/jump bugs

DV Checks Using pc_out

- PC increments by 4
- PC updates correctly on branch/jump
- PC always 4-byte aligned

Example assertion: assert ((pc_out & 32'h3) == 0);

6 Register File Output (regs_out[0:31])

What is regs_out?

logic [31:0] regs_out [0:31];

Exposes all **32 RISC-V architectural registers**:

x0 → hardwired zero

x1–x31 → general purpose

Why Expose Registers for DV?

To verify:

ALU results

Immediate handling

Load writeback

Destination register selection

Example

Instruction: ADD x3, x1, x2

DV check: `regs_out[3] == regs_out[1] + regs_out[2]`

Special Rule: x0 Register

RISC-V rule: x0 must always be zero

DV assertion: `assert (regs_out[0] == 0);`

Catches : Illegal writes, Writeback bugs

7 Memory Verification Rule (★)

DV Rule #1 : Only verify memory when an instruction accesses memory.

Memory is checked:

- On stores → update expected memory
- On loads → compare expected vs actual

Memory is verified indirectly, not continuously

8 Clocking Block & Signal Direction

Clocking blocks Purpose :

Synchronize TB & DUT

Control sampling & driving timing

Improve assertion reliability

Important: Clocking directions are from testbench perspective only.

Signal Ownership

CPU Drives:

`imem_addr`, `dmem_addr`, `dmem_wdata`, `dmem_wstrb`, `dmem_we`

Testbench Drives:

`imem_rdata`, `dmem_rdata`

TB reads everything else.

DV Checks Using Clocking Block

- x0 never changes
 - PC alignment
 - Valid store strobes
 - Load/store correctness
 - Opcode coverage
-

9 Assertions (SVA)

1 x0 Must Always Be Zero

```
assert property (@(posedge clk) disable iff(reset)
    regs_out[0] == 0);
```

2 PC Must Be Aligned

```
assert ((pc_out & 32'h3) == 0);
```

3 No Memory Writes During Reset

```
assert property (@(posedge clk) reset |-> !dmem_we);
```

4 Valid Store Strobe

```
assert property (@(posedge clk)
    dmem_we |-> (dmem_wstrb != 0));
```

10 Transaction Items

Why Transaction Items Exist

They:

- Represent instructions and memory ops
- Carry expected & observed values
- Help scoreboard compare behavior

■ They DO NOT drive hardware

i. riscv_txn — Instruction Transaction

rand bit [31:0] instr;

rand bit [31:0] pc;

bit [4:0] rs1, rs2, rd;

bit [31:0] op1, op2, alu_res;

bit branch_taken;

Key Insight :

- instr → stimulus
- pc → label/context (NOT driving DUT)

Think: pc here = tag, not control

ii. mem_txn — Memory Transaction

rand bit [31:0] addr;

rand bit [31:0] wdata;

rand bit [3:0] wstrb;

rand bit is_write;

bit [31:0] rdata;

Used to:

- Track loads/stores
- Compare memory behavior

Prediction Flow (Golden DV Concept)

1. TB generates instruction
2. Reference model executes instruction
3. Expected PC / regs / memory updated
4. DUT executes instruction
5. Monitor captures actual behavior
6. Scoreboard compares expected vs actual

Match → PASS

Mismatch → BUG

1 2 Final Golden Rule

“Transaction items do not control hardware. They carry expected and observed architectural information so the scoreboard can validate DUT behavior against a golden reference model.”

Final Insights

In real hardware:

- Instructions live in memory
- CPU fetches using PC

In verification:

- No real instruction memory exists
- TB models instruction memory behavior
- CPU generates PC
- TB responds with instruction for that PC
- Any mismatch = control flow bug